

UML - 01 - Diagramma dei casi d'uso

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



- 1 Introduzione UML
- 2 Diagramma Casi d'uso

Argomenti trattati

- 1 Introduzione UML
- 2 Diagramma Casi d'uso

Cos'è UML?

- Il Unified Modeling Language (UML) è un linguaggio di **modellazione visiva** utilizzato per rappresentare in modo chiaro e standardizzato i diversi aspetti di un sistema SW.
- UML permette di descrivere i requisiti, le strutture dati, i flussi di informazioni e i processi interni di un sistema, utilizzando un approccio basato su concetti orientati agli oggetti.
- È il risultato della consolidazione delle best practice sviluppate negli anni per la modellazione dei sistemi SW complessi.

Motivazione

- Lo sviluppo di un SW richiede una comunicazione chiara tra sviluppatori, analisti e clienti.
- Il linguaggio naturale è spesso ambiguo e può portare a interpretazioni errate dei requisiti.
- UML fornisce un metodo strutturato per descrivere il sistema in una forma visiva, riducendo il rischio di incomprensioni e facilitando la collaborazione tra le parti coinvolte.

Linguaggi di Modellazione

- I linguaggi di modellazione servono a rappresentare sistemi in modo strutturato attraverso regole e convenzioni precise.
- Possono essere testuali (come i linguaggi di programmazione) o visuali (come schemi di circuiti elettronici o diagrammi UML).
- Alcuni linguaggi sono specifici per determinati domini (es. modellazione di applicazioni web), mentre altri, come UML, sono general-purpose.

Concetto di Modello

- Un modello è una rappresentazione semplificata di un sistema, utile per analizzare, progettare e documentare il funzionamento del SW.
- Un sistema è un insieme di componenti interconnessi con uno scopo preciso.
- Esempi di sistemi: automobili, ecosistemi, aziende e SW.

Storia ed Evoluzione di UML

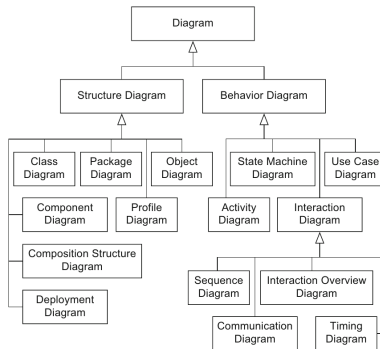
- UML è stato introdotto nella versione 1.0 nel 1997 come standard per la modellazione SW.
- Deriva dall'unione di tre metodologie principali: Booch, Object Modeling Technique (OMT) e Object-Oriented Software Engineering (OOSE).
- Nel tempo è stato aggiornato con nuove versioni, diventando lo standard de facto per la modellazione dei sistemi SW.

Caratteristiche di UML

- UML è indipendente da specifici strumenti di sviluppo, linguaggi di programmazione o piattaforme.
- Non impone un particolare processo di sviluppo, ma si integra bene con approcci iterativi e incrementali.
- Il metamodel UML definisce gli elementi costitutivi del linguaggio e il loro utilizzo corretto.

Diagrammi UML

- I modelli UML sono rappresentati tramite diagrammi grafici.
- Un diagramma fornisce una vista specifica del sistema modellato, evidenziando strutture, interazioni o comportamenti.



- UML 2.4.1 offre 14 tipi di diagrammi suddivisi in due categorie principali:
 - **Diagrammi strutturali:** rappresentano gli aspetti statici del sistema.
 - **Diagrammi comportamentali:** descrivono il flusso di operazioni e le interazioni tra gli elementi.

Vantaggi dell'Uso di UML

- Fornisce un linguaggio standard comprensibile a sviluppatori, analisti e stakeholder.
- Facilita la comunicazione e la documentazione durante il ciclo di vita del SW.
- Supporta la progettazione di sistemi complessi tramite un insieme strutturato di diagrammi.
- Aiuta a ridurre errori e ambiguità nella definizione dei requisiti e nella progettazione del sistema.

Limitazioni di UML

- La flessibilità di UML può portare a interpretazioni diverse dello stesso modello.
- L'uso di UML può essere complesso e richiede una conoscenza approfondita delle sue regole e convenzioni.
- Gli strumenti di modellazione UML possono implementare il linguaggio in modi leggermente differenti, rendendo difficile l'interoperabilità tra di essi.
- Tanti SW in circolazione:
 - Potenti e facili da usare (a pagamento)
 - Usabili con meno funzionalità (open source - free)

Software di modellazione UML opensource

- Modelio (GUI, multiplatforma): <https://www.modelio.org/>
- Umbrello (GUI, linux KDE): <https://uml.sourceforge.io/>
- PlantUML (testuale): <https://plantuml.com/>

Argomenti trattati

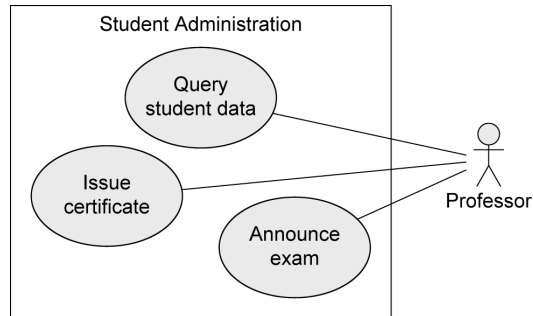
- 1 Introduzione UML
- 2 Diagramma Casi d'uso

Casi d'uso: introduzione

- Il caso d'uso è un concetto fondamentale di molti metodi di sviluppo orientati agli oggetti.
- I diagrammi dei casi d'uso esprimono le aspettative dei clienti/stakeholder.
 - essenziale per una progettazione dettagliata
- Il diagramma dei casi d'uso viene utilizzato durante l'intero processo di analisi e progettazione.
- Possiamo utilizzare un diagramma dei casi d'uso per rispondere alle seguenti domande:
 - Cosa viene descritto? (Il **sistema**).
 - Chi interagisce con il sistema? (Gli **attori**).
 - Cosa possono fare gli attori (I **casi d'uso**).

Esempio: Sistema di amministrazione degli studenti

- **Sistema:** cosa viene descritto?
 - Sistema di amministrazione degli studenti
- **Attori:** chi interagisce con il sistema?
 - Professore
- **Casi d'uso:** cosa possono fare gli attori?
 - Interrogare i dati degli studenti
 - Rilascio del certificato
 - Annuncio dell'esame

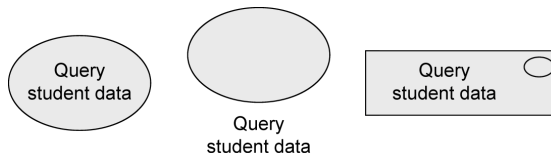


Caso d'uso

- Simbolo



- Descrive le funzionalità attese dal sistema in fase di sviluppo.
- Fornisce un beneficio tangibile a uno o più attori che comunicano con questo caso d'uso.
- Derivato dai desideri dei clienti raccolti durante le varie fasi.
- L'insieme di tutti i casi d'uso descrive le funzionalità che un sistema deve fornire.
 - Documenta le funzionalità offerte da un sistema.
- Notazioni alternative:

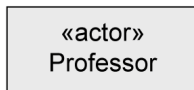
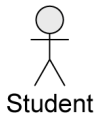


Attore (1/3)

- Simbolo

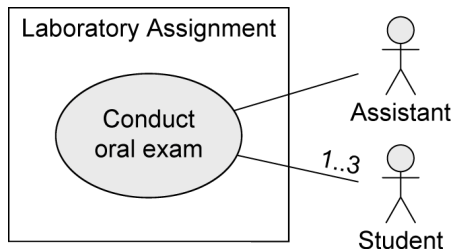


- Gli attori interagiscono con il sistema in due modi:
 - avviando l'esecuzione dei casi d'uso;
 - fornendo funzionalità necessarie per l'esecuzione dei casi d'uso.
- Gli attori rappresentano i ruoli che gli utenti adottano.
 - Utenti specifici possono adottare e accantonare più ruoli contemporaneamente.
- Gli attori non fanno parte del sistema, cioè sono al di fuori dei confini del sistema.
- Notazioni alternative:



Attore (2/3)

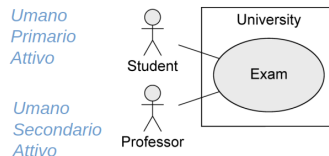
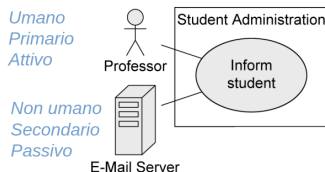
- Di solito anche i dati degli utenti vengono amministrati all'interno del sistema. Questi dati sono modellati all'interno del sistema sotto forma di oggetti e classi.
- Esempio: l'attore `Assistant`
 - L'attore `Assistant` interagisce con il sistema `Laboratory Assignment` utilizzandolo.
 - La classe `Assistant` descrive gli oggetti che rappresentano i dati dell'utente (es: nome, cognome, ...).



Attore (3/3)

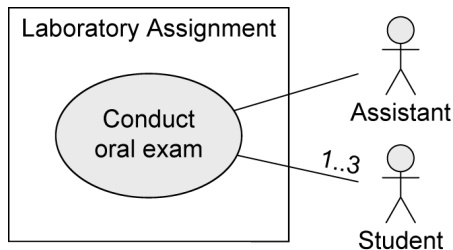
- Umano
 - Ad esempio: Student, Professor
- Non umano
 - Ad esempio: E-Mail, Server
- Primario: ha il beneficio principale dell'esecuzione del caso d'uso.
- Secondario: non riceve alcun beneficio diretto
- Attivo: avvia l'esecuzione del caso d'uso.
- Passivo: fornisce funzionalità per l'esecuzione del caso d'uso.

Esempio:



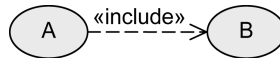
Relazioni tra Casi d'uso e Attori

- Gli attori sono collegati ai casi d'uso tramite linee solide (associazioni).
- Ogni attore deve comunicare con almeno un caso d'uso.
- Un'associazione è sempre binaria.
- È possibile specificare le molteplicità.

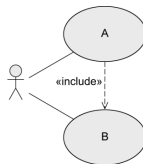


Relazioni tra i casi d'uso: Relazione "include"

- Simbolo



- Il comportamento di un caso d'uso (caso d'uso incluso) è integrato nel comportamento di un altro caso d'uso (caso d'uso base).



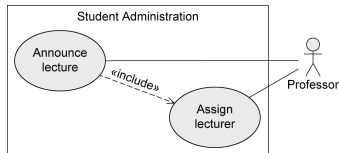
Caso d'uso base

richiede il comportamento dell'uso incluso per poter offrire le sue funzionalità

Caso d'uso incluso

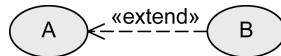
può essere eseguito da solo

- Esempio:

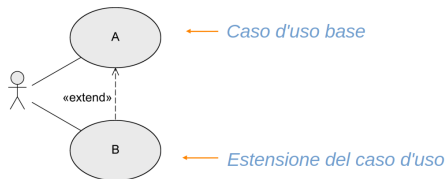


Relazioni tra i casi d'uso: Relazione "extend"

- Simbolo



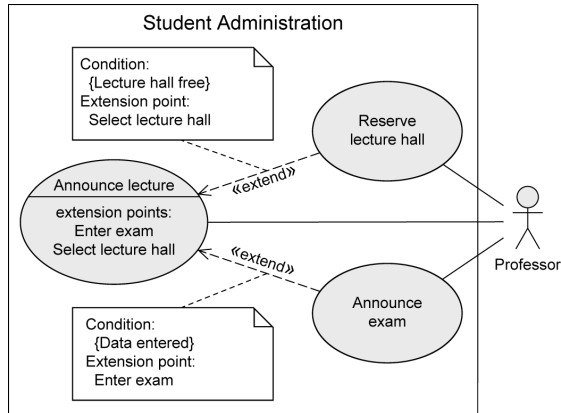
- Il comportamento di un caso d'uso (caso d'uso esteso) può essere integrato nel comportamento di un altro caso d'uso (caso d'uso base), ma non necessariamente.
- Entrambi i casi d'uso possono essere eseguiti **anche** indipendentemente l'uno dall'altro.



- A decide se B viene eseguito.
- I **punti di estensione** definiscono in quale punto il comportamento viene integrato.
- Cioè, sono condizioni che definiscono in quali circostanze il comportamento viene integrato.

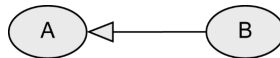
Relazioni tra i casi d'uso: Relazione "extend", punti di estensione

- I punti di estensione sono scritti direttamente all'interno del caso d'uso.
- È possibile specificare più punti di estensione.
- Esempio:

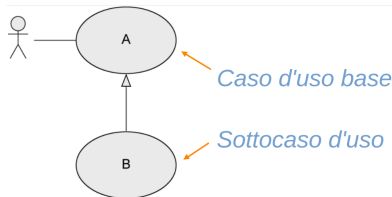


Relazioni tra i casi d'uso: Generalizzazione dei casi d'uso

- Simbolo

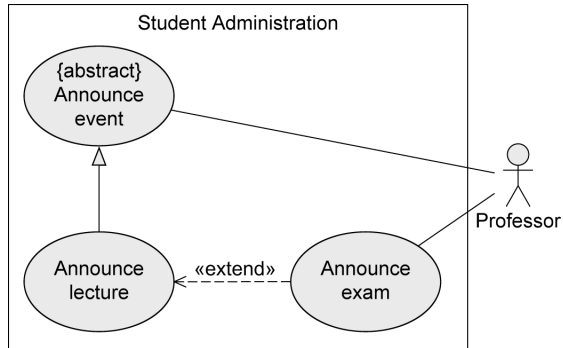


- Il caso d'uso **A** generalizza il caso d'uso **B**.
- B eredita il comportamento di A e può estendere o sovrascrivere.
- B eredita anche tutte le relazioni da A.
- B adotta la funzionalità di base di A ma decide autonomamente quale parte di A viene eseguita o modificata.
- A può essere etichettato come *abstract*
 - Non può essere eseguito direttamente
 - Solo B è eseguibile



Relazioni tra i casi d'uso: Generalizzazione dei casi d'uso (2)

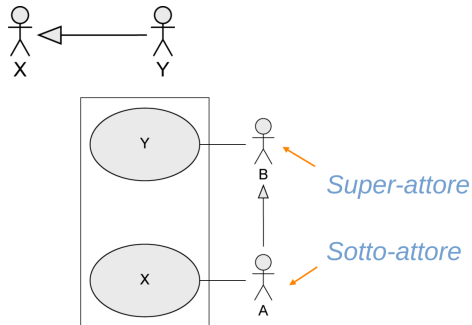
- Esempio:



Relazioni tra attori: Generalizzazione degli attori

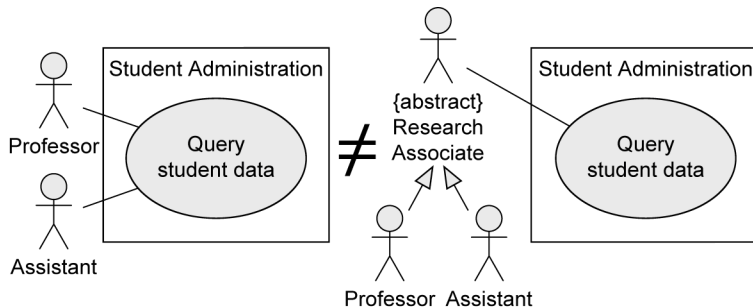
- Simbolo

- L'attore A eredita dall'attore B.
- A può comunicare con X e Y.
- B può comunicare solo con Y.
- È consentita l'ereditarietà multipla.
- Sono possibili attori astratti.



Relazioni tra attori: Generalizzazione degli attori (2)

Esempio:



Professor **AND** Assistant necessari
per eseguire Query student data

Professor **OR** Assistant necessari
per eseguire Query student data

Descrizione dei casi d'uso

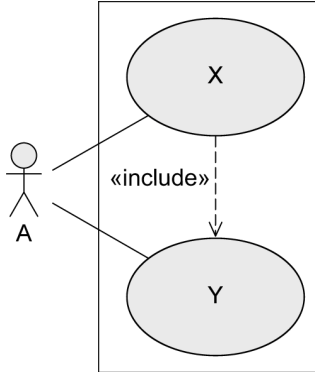
- Generalmente 1-2 pagine di documentazione per ciascun caso d'uso
- Approccio strutturato
 - Nome
 - Breve descrizione
 - Precondizione: prerequisito per un'esecuzione corretta
 - Postcondizione: stato del sistema dopo l'esecuzione
 - Situazioni di errore: errori rilevanti per il dominio del problema
 - Stato del sistema al verificarsi di un errore
 - Attori che comunicano con il caso d'uso
 - Trigger: eventi che avviano il caso d'uso.
 - Processo standard: singoli passi da compiere
 - Processi alternativi: deviazioni dal processo standard

Descrizione dei casi d'uso - Esempio

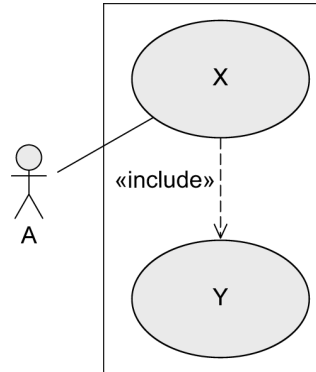
- Nome: Reserve lecture hall
- Breve descrizione: Un dipendente prenota un'aula dell'università per un evento.
- Precondizione: Il dipendente è autorizzato a prenotare le aule.
- Postcondizione: È stata prenotata un'aula.
- Situazioni di errore: Non c'è un'aula libera.
- Stato del sistema in caso di errore: Il dipendente non ha prenotato un'aula.
- Attori: Employee
- Trigger: Il dipendente richiede un'aula.
- Processo standard:
 - (1) Il dipendente accede al sistema.
 - (2) Il dipendente seleziona l'aula.
 - (3) Il dipendente seleziona la data.
 - (4) Il sistema conferma che l'aula è libera.
 - (5) Il dipendente conferma la prenotazione.
- Processi alternativi:
 - (4') L'aula non è libera.
 - (5') Il sistema propone un'aula alternativa.
 - (6') Il dipendente seleziona un'aula alternativa e conferma la prenotazione.

Best practice: "include"

Standard UML

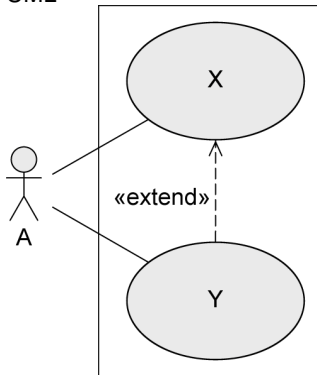


Best practice

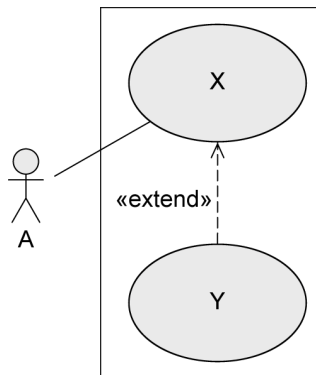


Best practice: "extend"

Standard UML



Best practice



Best practice

Identificazione degli attori

- Chi utilizza i principali casi d'uso?
- Chi ha bisogno di supporto per il proprio lavoro quotidiano?
- Chi è responsabile dell'amministrazione del sistema?
- Quali sono i dispositivi/sistemi SW esterni con cui il sistema deve comunicare?
- Chi è interessato ai risultati del sistema?

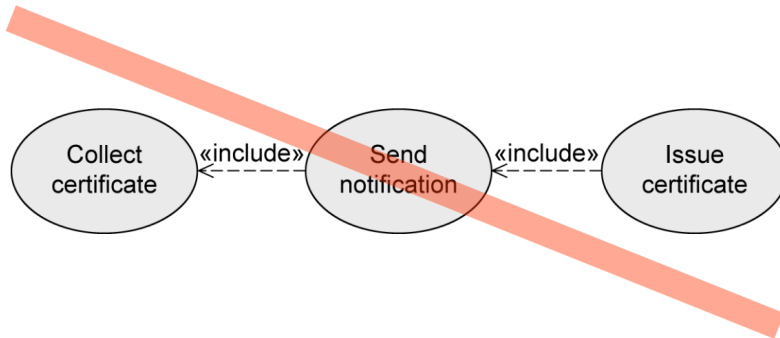
Best practice

Identificazione dei casi d'uso

- Quali sono i compiti principali che un attore deve svolgere?
- Un attore vuole interrogare o modificare le informazioni contenute nel sistema?
- Un attore vuole informare il sistema sui cambiamenti di altri sistemi?
- Un attore deve essere informato di eventi inaspettati all'interno del sistema?

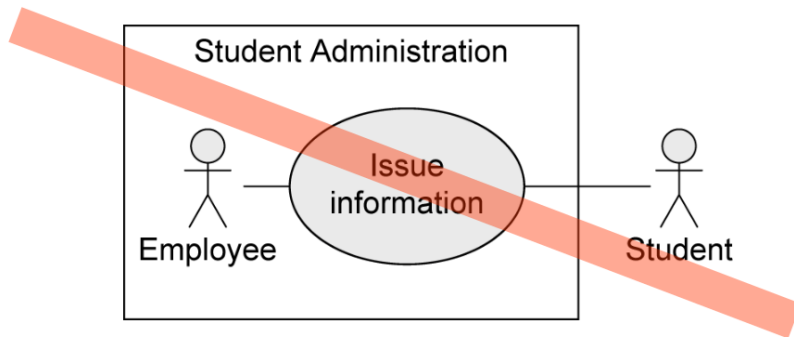
Errori tipici da evitare (1/5)

- I diagrammi dei casi d'uso non modellano processi/flussi di lavoro!



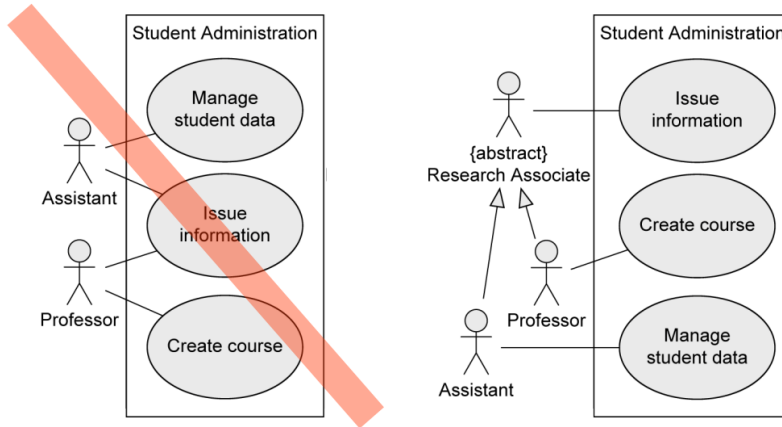
Errori tipici da evitare (2/5)

- Gli attori non fanno parte del sistema, quindi sono posizionati al di fuori dei confini del sistema!



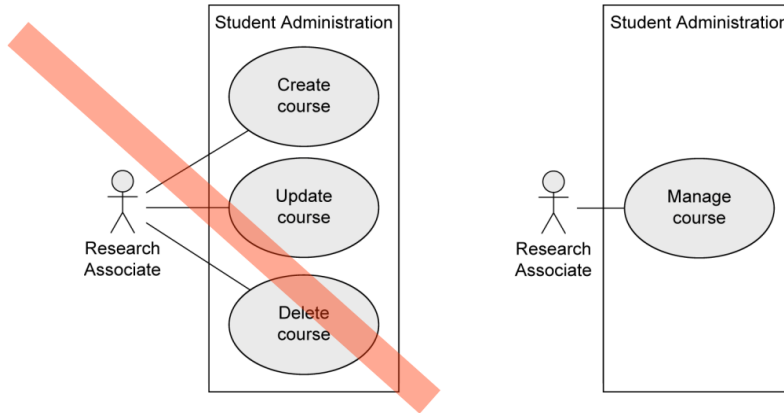
Errori tipici da evitare (3/5)

- **Caso d'uso** *Issue information* necessita o di un attore *Assistant* o di un attore *Professor* per l'esecuzione.



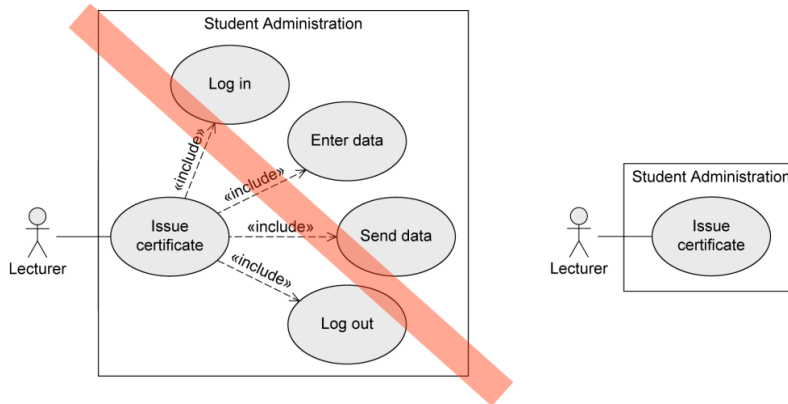
Errori tipici da evitare (4/5)

- Molti casi d'uso piccoli che hanno lo stesso obiettivo possono essere raggruppati per formare un unico caso d'uso più grande.

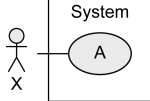


Errori tipici da evitare (5/5)

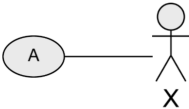
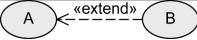
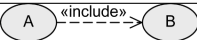
- Le varie fasi fanno parte dei casi d'uso, non sono casi d'uso separati! -> NO decomposizione funzionale



Elementi di notazione (1/2)

Nome	Notazione	Descrizione
Sistema		Confini tra il sistema e gli utenti del sistema
Caso d'uso		Unità di funzionalità del sistema
Attore		Ruolo degli utenti del sistema

Elementi di notazione (2/2)

Nome	Notazione	Descrizione
Associazione		Relazione tra casi d'uso e attori
Generalizzazione		Relazione di ereditarietà tra attori o casi d'uso
Estendere la relazione		B estende A: uso opzionale dell'uso caso B dal caso d'uso A
Includere la relazione		A include B: uso richiesto di uso caso B dal caso d'uso A